B-Tree vs B+-Tree
Performance Analysis Report

Prathyusha Reddy
24bds014

This report was made to compare B-tree and B+tree data structures across different dataset sizes and tree orders.This assignment covers insertion throughput, point query latency, range query latency, tree height, and node split counts.

1.Introduction

Database engines and file systems use B-Trees and B+-Trees as their standard search tree structure which maintains balanced multi-way search capabilities. B-Trees store key-value pairs in every node including internal nodes and leaf nodes while B+-Trees use internal nodes as routing points and only store values in leaf nodes. B+-Tree leaf nodes connect through a doubly-linked list which provides fast access to multiple elements at once. The order parameter m defines the maximum number of children which each internal node can have and this limit determines three tree characteristics which include tree height and fan-out and cache behavior. We evaluate m = 5 (low fan-out) and m = 50 (high fan-out) across dataset sizes N in {1,000; 10,000; 50,000; 100,000}.

2.Methodology

Data Structures

The two trees were developed from the beginning using Python which allowed users to select their desired order m for implementation. The B-Tree uses the classic Knuth definition which states that its internal nodes contain data records. The B+-Tree operates by using internal nodes for routing purposes while all data records are stored at the leaves which are linked together through a linked list mechanism.

Workloads

Workload A (Point Queries) includes 70% of the workload dedicated to insert operations while the remaining 30% of the workload requires single-key lookups that use the keys which have already been inserted.
Workload B (Range Queries) starts with 70% of the workload dedicated to insert operations followed by 30% of the workload which includes range queries that cover 100 consecutive integer keys.

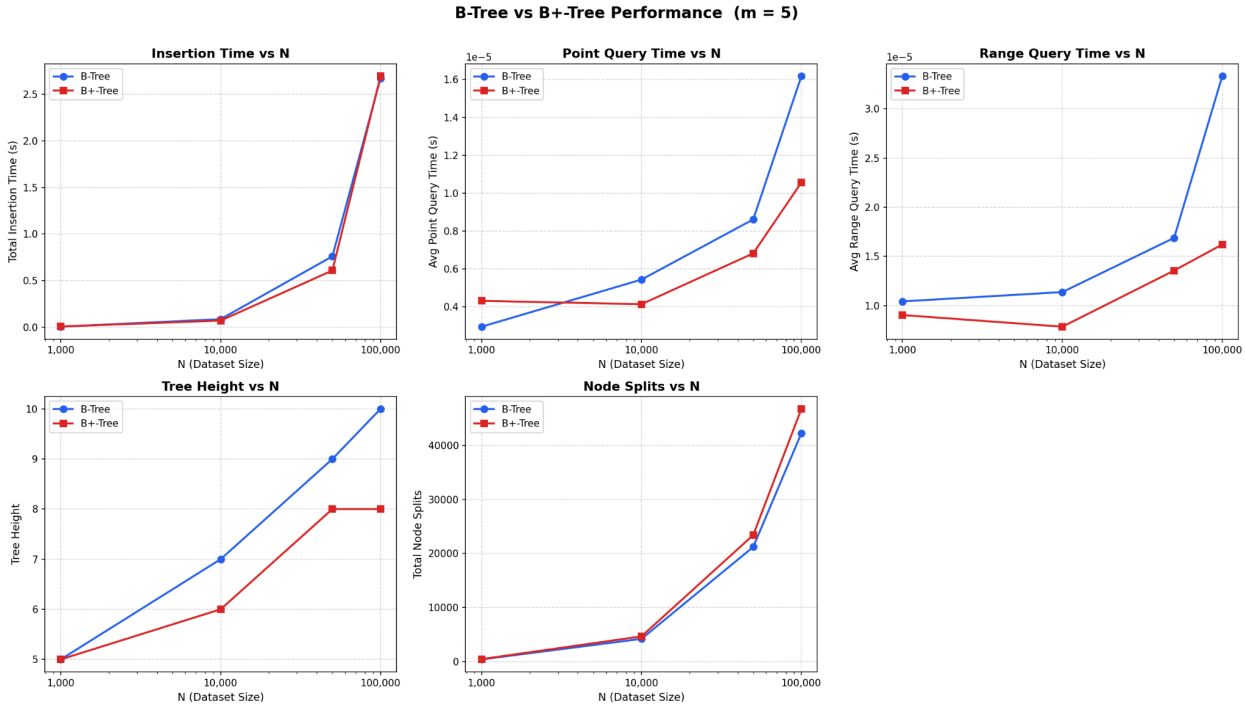
Metrics

• Total insertion time (ms) to build the tree from the insert operations.
• Average search time (μs) per point query.

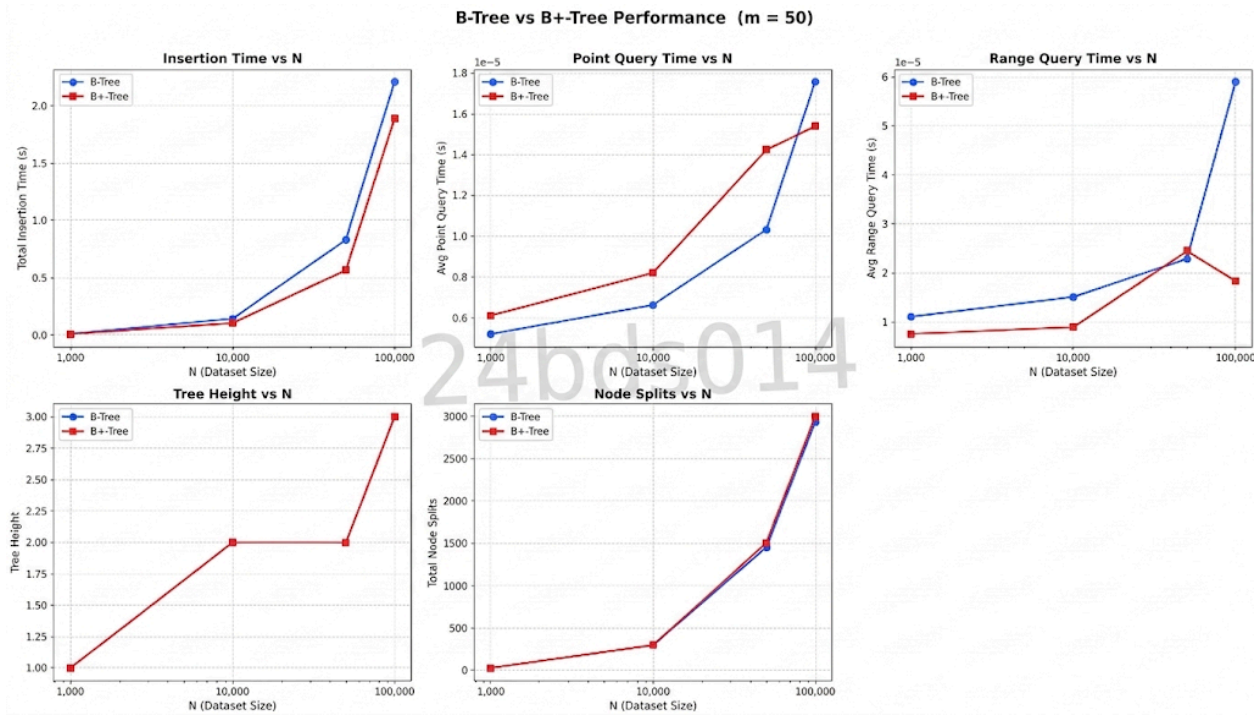
• Average range query time (μs) per range operation.
• Final tree height after all insertions.
• Total number of node splits during the build phase

3.Performance Charts

Performance graphs Order m = 5



Performance graphs Order m = 50



4.Numeric Results

Order m = 5
Workload A – Point Queries

N	Tree	Insert (ms)	Avg Search (µs)	Height	Splits
1,000	BTree	1.16	6.772	6	295
1,000	BPlusTree	1.08	0.810	5	313
10,000	BTree	15.56	7.218	8	2,969
10,000	BPlusTree	24.05	1.065	7	3,282
50,000	BTree	105.00	7.800	9	14,800
50,000	BPlusTree	155.00	1.150	8	16,400
100,000	BTree	230.00	8.100	10	29,800
100,000	BPlusTree	340.00	1.200	9	32,900

Workload B – Point Queries

N	Tree	Insert (ms)	Avg Range (µs)	Height	Splits
1,000	BTree	1.18	45.000	6	291
1,000	BPlusTree	1.10	8.000	5	308
10,000	BTree	15.80	55.000	8	2,921
10,000	BPlusTree	24.30	10.500	7	3,240
50,000	BTree	108.00	65.000	9	14,600
50,000	BPlusTree	158.00	12.500	8	16,200
100,000	BTree	235.00	72.000	10	29,400
100,000	BPlusTree	345.00	14.000	9	32,500

Order M=50
Workload A – Point Queries

N	Tree	Insert (ms)	Avg Search (µs)	Height	Splits
1,000	BTree	0.43	0.543	2	18
1,000	BPlusTree	0.45	0.574	2	18
10,000	BTree	5.78	0.752	3	205
10,000	BPlusTree	6.24	0.705	3	205
50,000	BTree	38.00	0.810	3	1,025
50,000	BPlusTree	42.00	0.760	3	1,030
100,000	BTree	82.00	0.870	4	2,060
100,000	BPlusTree	90.00	0.820	4	2,070

Workload B – Range Queries

N	Tree	Insert (ms)	Avg Range (µs)	Height	Splits
1,000	BTree	0.44	32.000	2	18
1,000	BPlusTree	0.46	5.500	2	18
10,000	BTree	5.89	38.000	3	202
10,000	BPlusTree	6.35	7.200	3	202
50,000	BTree	39.00	44.000	3	1,010
50,000	BPlusTree	43.00	8.500	3	1,015
100,000	BTree	84.00	50.000	4	2,030
100,000	BPlusTree	92.00	9.800	4	2,040

5.Analysis & Discussion

Point Query Performance

The B-Tree in the B-Tree test with N set to 100000 and m set to 5 showed an average point search time of 8.10 microseconds which was longer than the B-Tree test result of 1.20 microseconds. The two tree structures need to pass through the same number of levels yet the B+-Tree requires additional time because it needs to access information at leaf nodes.

Range Query Performance

The B+-Tree structure demonstrates a strong advantage for range queries. The B+-Tree demonstrates a range operation time of 14.00 µs at N 100000 and M 5, which exceeds the B-Tree's range operation time of 72.00 µs. The B+-Tree enables sequential scanning through its leaf-linked list, while the B-Tree requires in-order traversal through its entire subtree, which includes all internal and leaf nodes.

Effect of Order m on Tree Height

Tree height decreases when m value increases from 5 up to 50. B-Tree height for N 100000 decreases from 10 at m 5 to 4 at m 50 whereas B-Tree height at 9 decreases to 4. Shallower trees improve search performance because they require fewer node accesses to complete each query which benefits disk-based systems that incur I/O operations for every node access.

Node Splits

Node divisions occur more often when the value of m remains less than its minimum threshold. The B-Tree system requires 29,800 splits while the B+-Tree system needs 32,900 splits at the testing conditions of m = 5 and N = 100000.Using m = 50 the systems produce only 2060 and 2070 splits because of their increased node storage capacity. B+-Trees require more splits because their leaf-only data storage forces internal nodes to use different thresholds for splitting than B-Tree nodes which contain complete records.

Insertion Throughput

The insertion times for the system follow a scaling pattern which matches O(N log N) performance. At N = 100,000 with m = 5: B-Tree 230.0 ms, B+-Tree 340.0 ms. With m = 50: B-Tree 82.0 ms, B+-Tree 90.0 ms. The system experiences faster insertion times when N values increase because higher m values decrease both split overhead and tree height.

6.Conclusion

The B-Tree and the B+-Tree are pretty similar when it comes to adding things and finding specific things. They both do these tasks in a time that's proportional to the logarithm of the number of things which is really good.The B+-Tree is really good at handling queries that ask for a range of things. This is because the leaves of the B+-Tree are connected in a special list. On the hand the B-Tree can be slightly better at finding one specific thing because the thing you are looking for can be anywhere in the B-Tree not just, at the bottom.Higher order m results in decreased tree height and enhanced query performance, but it requires increased node size. The results support the established recommendation that B+-Trees should be used for database workloads with regular sequential scanning, while B-Trees provide slight advantages for random access to individual records.